Consulta 1 | Etapa 04

POSTGRESQL · EQUiDADOS

ANTI JOIN · LEFT JOIN + IS NULL

Objetivo. Localiza equipes ativas que não têm atendimentos simulados associados nos últimos 90 dias. O LEFT JOIN mantém equipes sem correspondência; o filtro IS NULL seleciona somente essas equipes.

Resultado esperado. Uma linha por equipe ativa sem atendimento no período, com unidade e distrito quando disponíveis.

SQL original de `Etapa_04_Consultas_Avancadas.sql`

```
-- CONSULTA 1 - ANTI JOIN (LEFT JOIN + IS NULL)
-- Equipes ativas sem atendimentos simulados nos ultimos 90 dias.
-- A data de corte e relativa ao dia de execucao.
SELECT e.codigo_equipe,
       e.nome_equipe,
       e.tipo_equipe,
       u.cnes AS cnes_unidade,
       COALESCE(u.nome_fantasia, u.nome_empresarial) AS unidade,
       u.distrito_sanitario_codigo
FROM equipes_saude e
LEFT JOIN unidades_saude u
    ON u.cnes = e.cnes_unidade
LEFT JOIN mock_pec_atendimentos a
    ON a.codigo_equipe = e.codigo_equipe
   AND a.data_atendimento >= CURRENT_DATE - INTERVAL '90 days'
   AND a.data_atendimento <= CURRENT_DATE
WHERE NULLIF(BTRIM(e.data_desativacao), '') IS NULL
  AND a.id_atendimento IS NULL
ORDER BY u.distrito_sanitario_codigo, e.codigo_equipe;
```

Consulta 2 | Etapa 04

POSTGRESQL · EQUiDADOS

FULL OUTER JOIN

Objetivo. Compara os distritos presentes no cadastro de unidades com os que aparecem nos atendimentos recentes. O FULL OUTER JOIN preserva distritos existentes em apenas uma das fontes.

Resultado esperado. Distrito, total de unidades, atendimentos recentes e indicação de quais fontes têm registros.

SQL original de `Etapa_04_Consultas_Avancadas.sql`

```
-- CONSULTA 2 - FULL OUTER JOIN
-- Compara distritos que aparecem na estrutura CNES com distritos que possuem
-- atendimentos simulados, mantendo categorias presentes em apenas uma fonte.
WITH unidades_por_distrito AS (
    SELECT distrito_sanitario_codigo,
           COUNT(DISTINCT cnes) AS total_unidades
    FROM unidades_saude
    WHERE distrito_sanitario_codigo IS NOT NULL
    GROUP BY distrito_sanitario_codigo
),
atendimentos_por_distrito AS (
    SELECT distrito_sanitario_codigo,
           COUNT(*) AS atendimentos_ultimos_90_dias
    FROM mock_pec_atendimentos
    WHERE data_atendimento >= CURRENT_DATE - INTERVAL '90 days'
      AND data_atendimento < CURRENT_DATE + INTERVAL '1 day'
      AND distrito_sanitario_codigo IS NOT NULL
    GROUP BY distrito_sanitario_codigo
)
SELECT COALESCE(
           u.distrito_sanitario_codigo,
           a.distrito_sanitario_codigo
       ) AS distrito_sanitario_codigo,
       COALESCE(u.total_unidades, 0) AS total_unidades,
       COALESCE(a.atendimentos_ultimos_90_dias, 0)
           AS atendimentos_ultimos_90_dias,
       CASE
           WHEN u.distrito_sanitario_codigo IS NULL
               THEN 'Somente atendimentos simulados'
           WHEN a.distrito_sanitario_codigo IS NULL
               THEN 'Somente estrutura CNES'
           ELSE 'Estrutura CNES e atendimentos simulados'
       END AS origem_dos_registros
FROM unidades_por_distrito u
FULL OUTER JOIN atendimentos_por_distrito a
    ON a.distrito_sanitario_codigo = u.distrito_sanitario_codigo
ORDER BY distrito_sanitario_codigo;
```

Consulta 3 | Etapa 04

POSTGRESQL · EQUiDADOS

SUBCONSULTA ESCALAR · MÉDIA

Objetivo. Conta os atendimentos por equipe e compara cada total com a média dos totais das equipes que têm registros. A subconsulta retorna um único valor: a média.

Resultado esperado. Equipes acima da média, ordenadas por volume decrescente de atendimentos.

SQL original de `Etapa_04_Consultas_Avancadas.sql`

```
-- CONSULTA 3 - SUBCONSULTA ESCALAR
-- Equipes com volume acima da media entre equipes com atendimentos.
WITH atendimentos_por_equipe AS (
    SELECT codigo_equipe, COUNT(*) AS total_atendimentos
    FROM mock_pec_atendimentos
    WHERE codigo_equipe IS NOT NULL
    GROUP BY codigo_equipe
)
SELECT e.codigo_equipe,
       e.nome_equipe,
       COALESCE(u.nome_fantasia, u.nome_empresarial) AS unidade,
       a.total_atendimentos
FROM atendimentos_por_equipe a
JOIN equipes_saude e ON e.codigo_equipe = a.codigo_equipe
LEFT JOIN unidades_saude u ON u.cnes = e.cnes_unidade
WHERE a.total_atendimentos > (
    SELECT AVG(media.total_atendimentos)
    FROM atendimentos_por_equipe media
)
ORDER BY a.total_atendimentos DESC, e.codigo_equipe;
```

Consulta 4 | Etapa 04

POSTGRESQL · EQUiDADOS

SUBCONSULTA ESCALAR · PROPORÇÃO

Objetivo. Calcula a proporção de registros Preta/Parda por distrito e a compara com a proporção municipal observada no conjunto simulado, calculada pela subconsulta escalar.

Resultado esperado. Distritos acima da proporção municipal observada, com contagens e percentual distrital. O percentual é baseado nos registros simulados.

SQL original de `Etapa_04_Consultas_Avancadas.sql`

```
-- CONSULTA 4 - SUBCONSULTA ESCALAR
-- Distritos cuja proporcao observada de registros pretos/pardos supera a
-- proporcao municipal observada nos atendimentos simulados.
WITH atendimentos_por_distrito AS (
    SELECT distrito_sanitario_codigo,
           COUNT(raca_cor) AS registros_com_raca_cor,
           COUNT(*) FILTER (
               WHERE raca_cor IN ('Preta', 'Parda')
           ) AS registros_pretos_ou_pardos
    FROM mock_pec_atendimentos
    WHERE distrito_sanitario_codigo IS NOT NULL
    GROUP BY distrito_sanitario_codigo
)
SELECT distrito_sanitario_codigo,
       registros_com_raca_cor,
       registros_pretos_ou_pardos,
       ROUND(
           100.0 * registros_pretos_ou_pardos
               / NULLIF(registros_com_raca_cor, 0),
           1
       ) AS percentual_preto_ou_pardo_distrital
FROM atendimentos_por_distrito
WHERE registros_com_raca_cor > 0
  AND 1.0 * registros_pretos_ou_pardos / registros_com_raca_cor > (
      SELECT 1.0 * COUNT(*) FILTER (
          WHERE raca_cor IN ('Preta', 'Parda')
      ) / NULLIF(COUNT(raca_cor), 0)
      FROM mock_pec_atendimentos
  )
ORDER BY percentual_preto_ou_pardo_distrital DESC;
```